

Problem Statement: Smart Inventory & Billing System

Design and implement a **Smart Inventory & Billing System** in Java for a small retail store. The system manages **products, customers, and billing**, while applying **various discount strategies**.

The solution must demonstrate:

- ✓ **OOP principles**
 - ✓ **Collections API**
 - ✓ **Java 8 features** (Streams, Optional, Lambdas, Functional Interfaces)
 - ✓ **Custom Exceptions**
 - ✓ **SOLID design principles**
 - ✓ **Design Patterns** (Factory, Strategy, Singleton)
 - ✓ **JUnit with TDD**
-

Requirements

1. Entities

- **Product**
 - Attributes: id, name, price, stockQuantity, category.
 - Products can be of different types (e.g., Grocery, Electronics, Clothing).
- **Customer**
 - Attributes: id, name, loyaltyPoints.
- **Bill**
 - Contains: List<Product> purchasedItems, totalAmount, discountApplied, finalAmount.

- **Cashier**
 - Generates bill for customer purchases.
-

2. Features

- Add, remove, and update products in inventory.
 - Search products by name/category using **Streams**.
 - Customers purchase items → system generates a bill.
 - Apply different discount strategies:
 - **PercentageDiscount** (e.g., 10% off).
 - **FixedDiscount** (e.g., ₹100 off).
 - **SeasonalDiscount** (context-based, e.g., Diwali sale).
 - Use `Optional<Product>` to handle product lookups safely.
 - Use `Lambda + FunctionalInterface` for custom filtering and discount logic.
-

3. Exception Handling

- `ProductNotFoundException` → if product doesn't exist.
 - `InsufficientStockException` → if stock < requested quantity.
 - `InvalidDiscountException` → if discount strategy is not applicable.
-

4. Design Principles (SOLID)

- **SRP** → Separate classes: InventoryService, BillingService, DiscountService.
 - **OCP** → New discount types can be added without changing existing code.
 - **LSP** → PercentageDiscount, FixedDiscount, SeasonalDiscount all implement DiscountStrategy.
 - **ISP** → Separate interfaces for Billable, Discountable.
 - **DIP** → High-level modules depend on abstractions (DiscountStrategy).
-

5. Design Patterns

- **Factory** → create different product types (Grocery, Electronics).
 - **Strategy** → different discount rules applied at runtime.
 - **Singleton** → Inventory is a singleton (one central stock).
-

6. JUnit with TDD

Write tests before coding:

- `testPurchaseWithPercentageDiscount()`
 - `testPurchaseWithFixedDiscount()`
 - `testOutOfStockThrowsException()`
 - `testSearchProductByCategoryUsingStream()`
 - `testOptionalForMissingProduct()`
-

7. Example Use Case

1. Inventory is initialized with products.
2. Customer selects products → cashier generates bill.
3. Discount strategy (e.g., **10% off**) is applied.
4. Stock is reduced in inventory.
5. If requested quantity > stock → `InsufficientStockException`.
6. If product not found → `ProductNotFoundException`.